



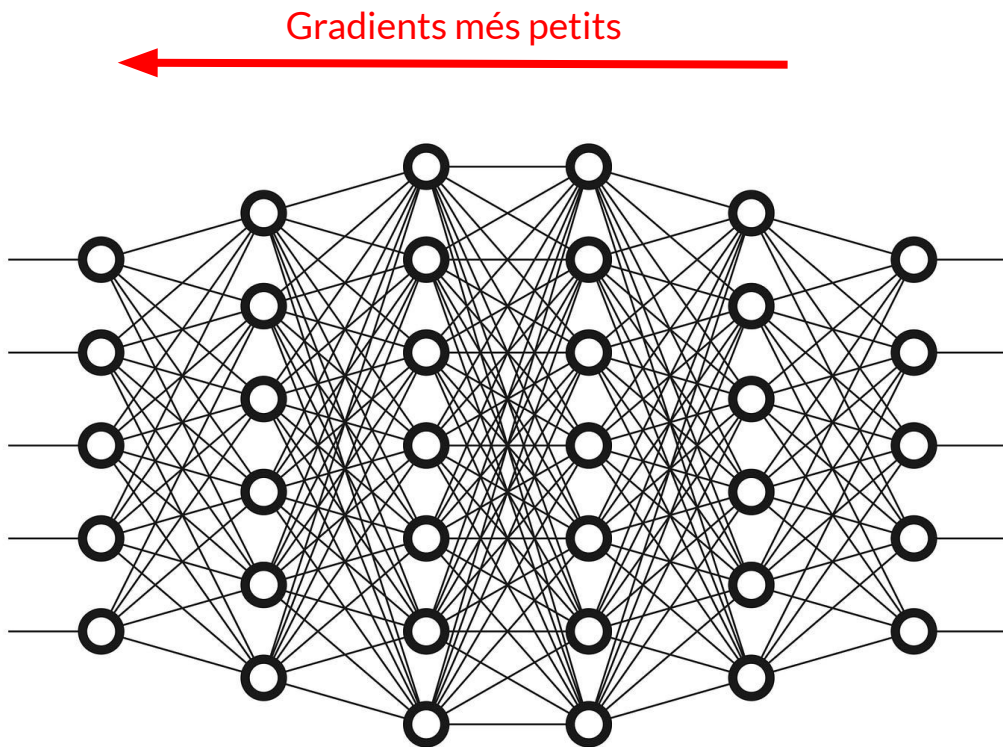
Més concepts de NNs



Problemes comuns amb els gradients

- Vanishing gradients.
- Exploding gradients.

Vanishing gradients





Vanishing gradients

- Síntomes:
 - L'aprenentatge triga molt o s'atura completament.
 - Els pesos més propers a les entrades no canvien gaire.
 - Els pesos es redueixen exponencialment des de l'última a la primera capa.



Vanishing gradients

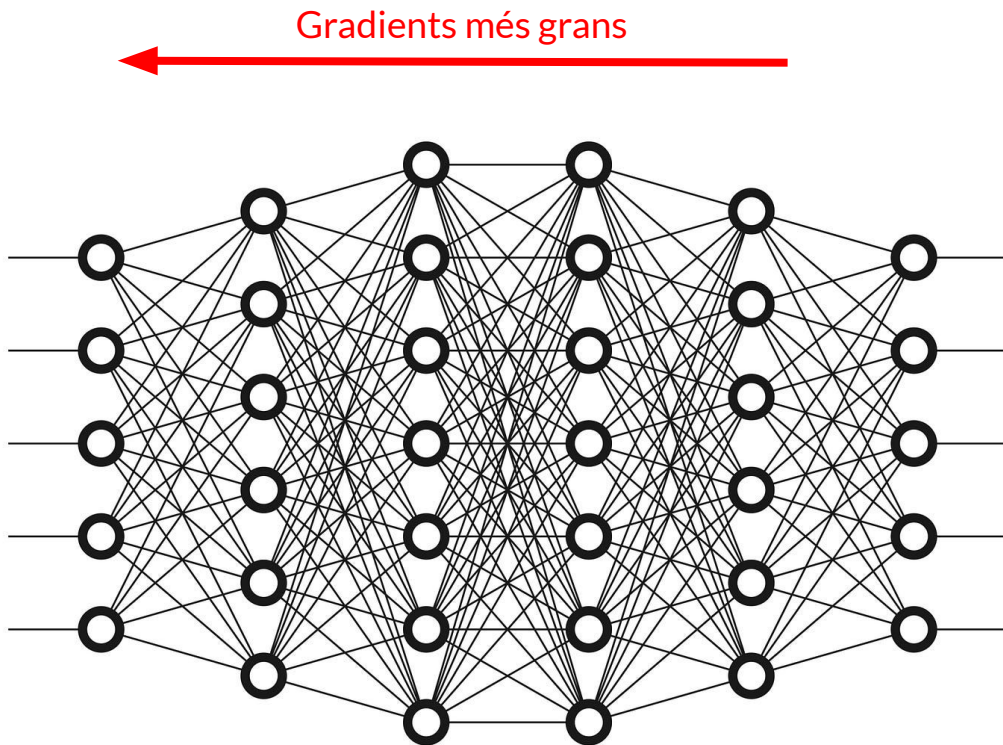
- Culpables:
 - Funcions d'activació que saturen, com la *sigmoid*.
 - Inicialització de pesos deficient.
 - Estructura de les RNN.
 - Xarxes molt profundes.



Vanishing gradients

- Solutions:
 - ReLU o similar.
 - Normalització per batches.
 - Gradient clipping.
 - Reduir el nombre de capes.

Exploding gradients





Exploding gradients

- Síntomes:
 - Creixement exponencial dels pesos des de l'última capa fins a la primera.
 - Desbordament dels pesos (NaN).
 - Els pesos més propers a les entrades són molt grans.



Exploding gradients

- Culpables:
 - Inicialització de pesos deficient.
 - Estructura de les RNN.
 - Xarxes molt profundes.



Exploding gradients

- Solucions:
 - Inicialització adequada dels pesos.
 - Normalització per batches.
 - Gradient clipping.
 - Reduir el nombre de capes.



Regularització (per evitar l'overfitting)

- Tècniques comunes:
 - Early stopping.
 - Dropout.
 - Batch normalization.
 - Data augmentation.



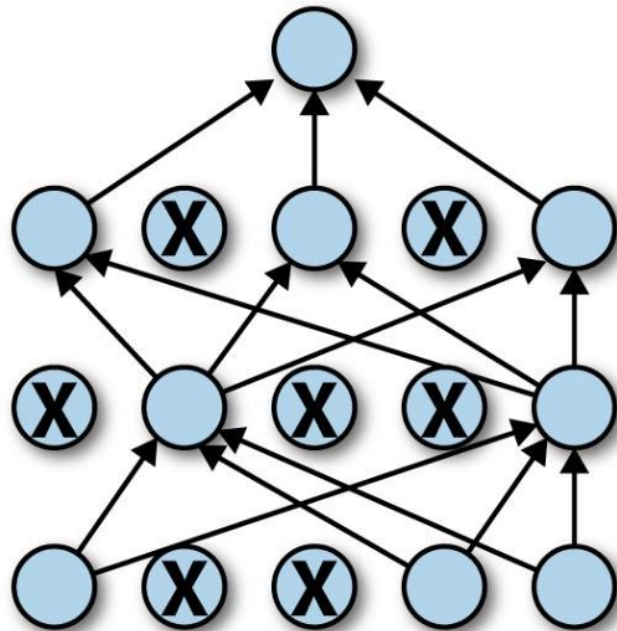
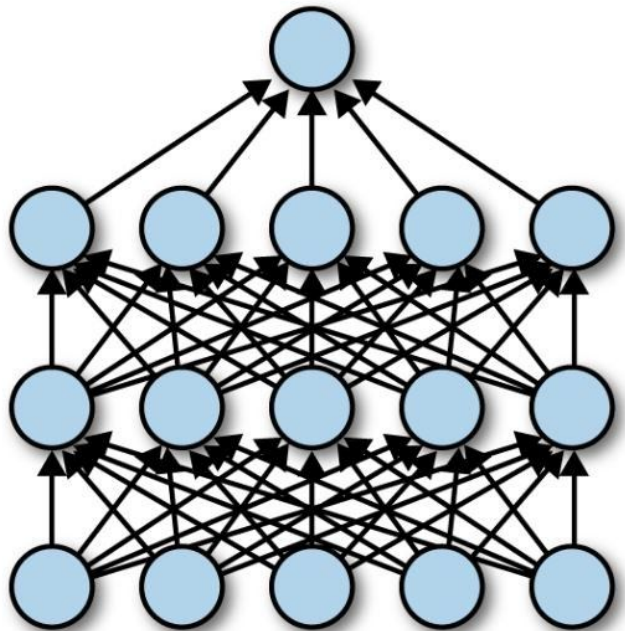
Dropout

- En cada iteració forward, es desactiva un subconjunt aleatori de neurones en una capa.
 - La seva sortida es posa a 0.
- Les neurones restants tenen els seus valors augmentats durant l'entrenament per mantenir l'activació total.
 - Augmentat multiplicant per: $1 / (1 - \text{dropout rate})$
- Durant la inferència (predicció) s'utilitzen totes les neurones sense dropout.

Dropout

- Beneficis:
 - Prevé l'overfitting.
 - Redueix la co-adaptació de les neurones.
 - Evita que les neurones es tornin “mandroses” / inactives (no contribueix gaire).
 - Força la xarxa a aprendre característiques més robustes i generalitzades.

Dropout





Dropout

- En quines capes?
 - Experimentació empírica...



Dropout – Recomanacions

- Taxa de dropout per capa:
 - FNNs simples: 10% a 30%
 - FNNs complexes: 40% o 50%
 - CNNs: 10% o 20%
- No necessàriament s'utilitza en cada capa.

Dropout – FNN Keras

```
nn_model = Sequential([
    Input(X_train.shape[1]),
    Dense(128, activation='relu'),
    Dropout(0.2),
    Dense(64, activation='relu'),
    Dropout(0.2),
    Dense(32, activation='relu'),
    Dense(len(class_names), activation='softmax')
])
```

- Assignar aleatòriament el 20% de les neurones a 0 en cada mini-batch.
- Aplicat a les capes Dense anteriors.

Dropout – CNN Keras

```
cnn_model = Sequential([
    Input(shape=(256, 256, 1)),
    Rescaling(1./255),
    Conv2D(filters=16, kernel_size=2, strides=1, activation='relu'),
    Dropout(0.1),
    MaxPooling2D(pool_size=2, strides=None),
    Conv2D(filters=32, kernel_size=2, strides=1, activation='relu'),
    Dropout(0.1),
    MaxPooling2D(pool_size=2, strides=None),
    Flatten(),
    Dense(32, activation='relu'),
    Dense(16, activation='relu'),
    Dense(len(class_names), activation='softmax')
])
```



Batch normalization

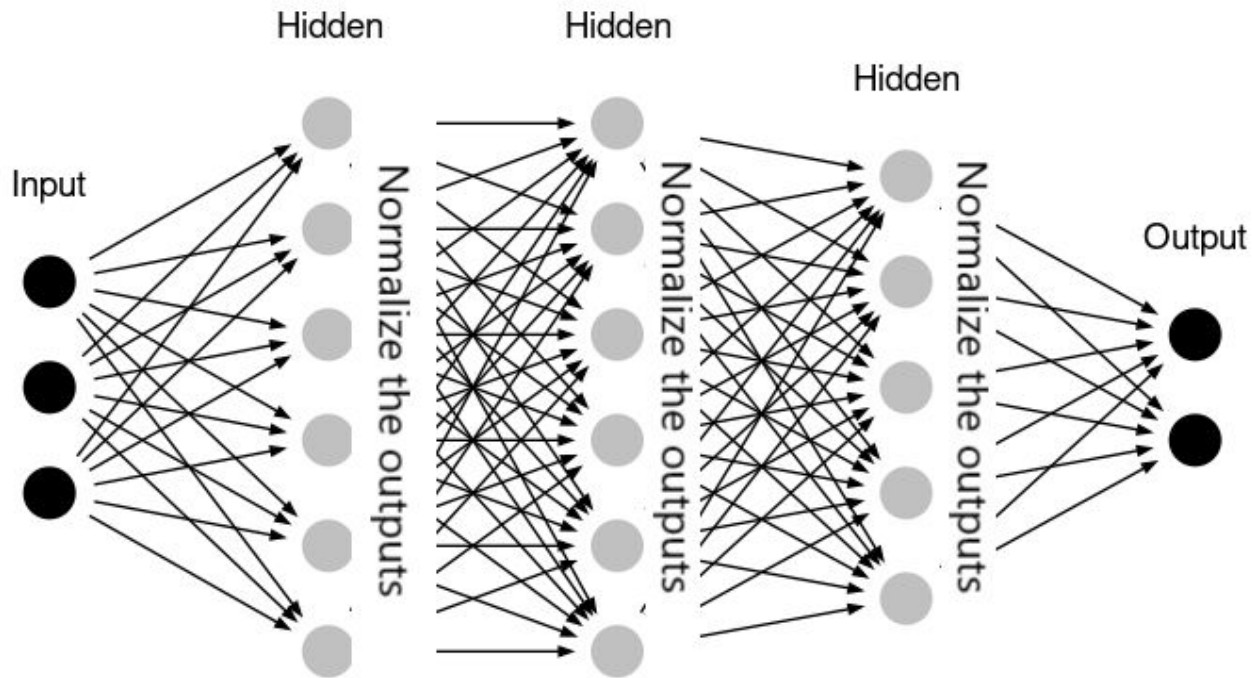
- Normalitza les activacions de les neurones en capes ocultes.
 - Mitjana zero i variància unitària basada en cada mini-batch en cada neurona.
- Això limita les activacions a un rang més consistent.



Batch normalization

- Beneficis:
 - Redueix les variacions en la distribució de l'activació dels neurones interns.
 - Permet que xarxes més profundes s'entrenin de manera més efectiva.
 - Permet learning rates més alts.

Batch normalization





Batch normalization

- En quines capes?
 - Moltes vegades s'utilitza a cada capa oculta (òbviament no a la sortida).
 - Normalització aplicada abans de la funció d'activació.



Capa BatchNormalization – Recomenacions

- No l'utilitzeu quan:
 - Mini-batches petits (p. ex., 4 o 8).
 - RNNs.

Capa BatchNormalization

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense, Dropout, BatchNormalization, Activation

model = Sequential([
    Input(shape=(1024,)),
    BatchNormalization(),

    # Hidden layer 1
    Dense(32),
    BatchNormalization(),
    Activation('relu'),
    Dropout(0.3),

    # Hidden layer 2
    Dense(16),
    BatchNormalization(),
    Activation('relu'),

    # Output
    Dense(6, activation='softmax')
])
```


Data agumentation



**Image
Augmentation**



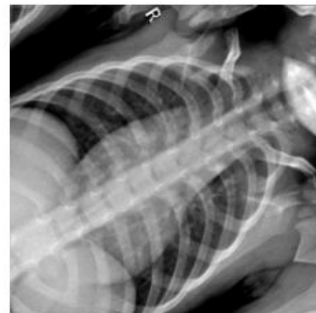
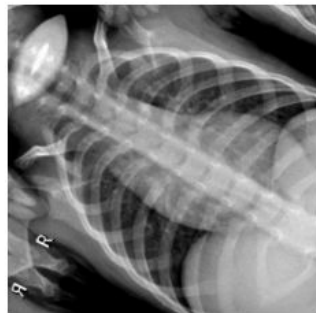
Data agumentation – Capes de Keras

- RandomCrop
- RandomFlip
- RandomTranslation
- RandomRotation
- RandomZoom
- RandomBrightness
- RandomContrast
- etc.

https://keras.io/api/layers/preprocessing_layers/image_augmentation/

Això s'aplica només durant l'entrenament.

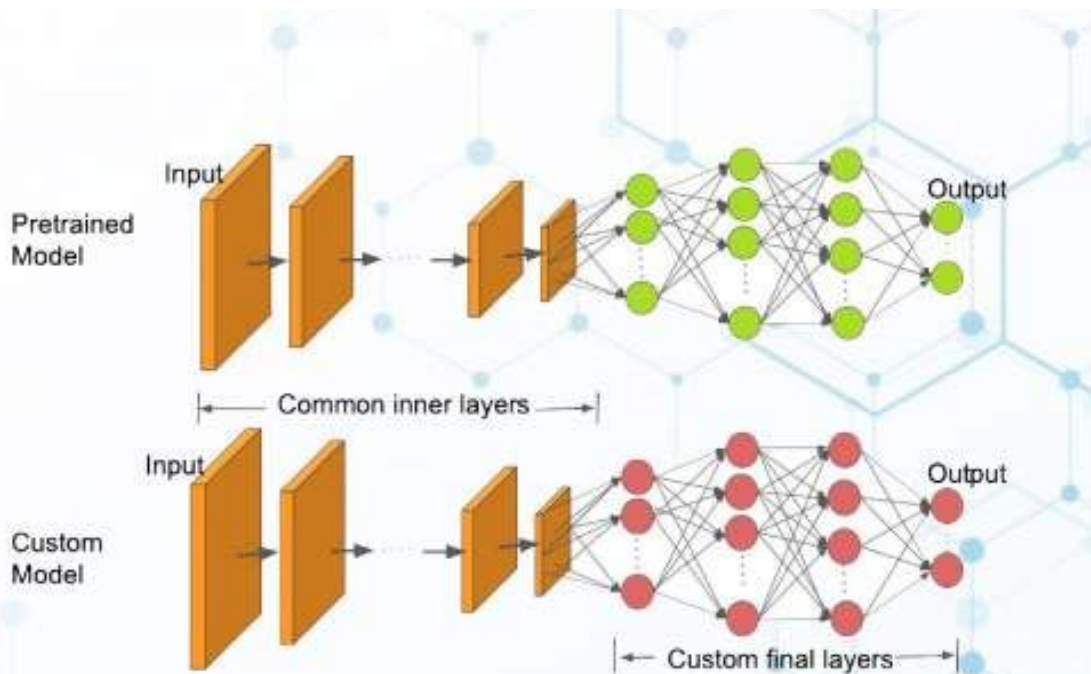
```
cnn_model = Sequential([
    Input(shape=(256, 256, 1)),
    Rescaling(1./255),
    layers.RandomFlip("horizontal_and_vertical"),
    layers.RandomRotation(0.2),
    Conv2D(filters=16, kernel_size=2, strides=1, activation='relu'),
    MaxPooling2D(pool_size=2, strides=None),
    Flatten(),
    Dense(32, activation='relu'),
    Dense(len(class_names), activation='softmax')
])
```



Data augmentation – Capes de Keras

- Imatges d'entrenament més diverses → potencialment millor generalització del model.
 - Pot requerir més temps d'entrenament.
- Trieu imatges transformades que siguin significatives i representatives.
 - No realitzeu qualsevol transformació aleatòria que us passi pel cap sense criteri.

Transfer Learning





Transfer learning

- Per adaptar la xarxa entrenada al nostre conjunt de dades, normalment només utilitzem la part CNN.
- Normalment es congelen els pesos de la CNN entrenada, per no canviar-los.

Transfer learning

- Si s'utilitzen imatges d'una mida diferent a la qual el model va ser entrenat originalment, es redimensionen sense respectar l'aspect ratio original.
 - Millor enfocament per a imatges més grans:
 1. Redimensiona basant-te en la dimensió més petita d'altura o amplada.
 2. Center-crop a la mida desitjada.
 - Si és més petita:
 - Aplica padding a les vores (normalment negre).

Transfer learning

- Pretrained CNNs: <https://keras.io/api/applications/>

Famous CNN architectures

<https://theaisummer.com/cnn-architectures/>

<https://keras.io/api/applications/>

```
from tensorflow.keras.applications import MobileNetV2
```

```
base_model = MobileNetV2(  
    weights='imagenet',  
    include_top=False,  
    # Not the default size, but we can use it anyway  
    input_shape=(256, 256, 3))
```

```
# Freeze all layers
```

```
for layer in base_model.layers:  
    layer.trainable = False
```

```
# Create a new sequential model
```

```
cnn_transf = Sequential([  
    Input(shape=(256, 256, 3)),  
    Rescaling(1./255),  
    base_model, # Add all the MobileNetV2 layers  
    # Add custom layers  
    Conv2D(32, kernel_size=2),  
    Conv2D(32, kernel_size=3),  
    Conv2D(32, kernel_size=3),  
    Dropout(0.1),  
    MaxPooling2D(pool_size=2),  
    Flatten(),  
    Dense(64, activation='relu'),  
    Dense(32, activation='relu'),  
    Dense(len(class_names), activation='softmax'),  
])
```

```
# Compile the model
```

```
cnn_transf.compile(optimizer=Adam(learning_rate=0.00003),  
                   loss='sparse_categorical_crossentropy',  
                   metrics=['accuracy'])
```